C++: Kelas dan Objek

IF2210 – Semester II 2025/2026

Sumber: Diktat Bahasa C++ oleh Hans Dulimarta

Modernisasi oleh Satrio Adi Rukmono

Motivasi PBO vs. Prosedural

Motivasi PBO

- Sistem perangkat lunak terdiri dari **entitas yang saling berinteraksi**
- Entitas memiliki:
  - identitas
  - keadaan (*state*)
  - perilaku (*behaviour*)
- Paradigma PBO muncul untuk **memodelkan entitas tersebut secara eksplisit**

PBO ≠ teknik bahasa

PBO = cara berpikir tentang sistem

Prosedural vs. Objek

Prosedural

- Data pasif
- Prosedur / fungsi mengolah data
- Alur kontrol terpusat

Objek

- Objek aktif
- Objek menjaga konsistensi
- Tanggung jawab terdistribusi

Ilustrasi

Prosedural

- *Stack*-nya disimpan di sini ya: 0x3872 (sebuah alamat memori).
- Ini prosedur-prosedurnya: peek(), push(), pop(), isEmpty().
- Tolong jangan pakai prosedur lain.
  - (Tapi pemrogram tetap bisa membuat & memakai prosedur lain)

Objek

- Hai, saya objek *stack*. Anda bisa minta saya untuk:
  - memberi tahu nilai “top”,
  - menambahkan item ke *stack* secara LIFO,
  - mengambil item yg ada di “top”,
  - memberi tahu apakah *stack* kosong.

Kolaborasi Antar Objek

- Sistem OO = jaringan objek yang saling berkolaborasi
- Objek:
  - meminta layanan objek lain
  - tidak memanipulasi *state* internal objek lain
- Komunikasi dilakukan melalui **interface perila**
  (kadang disebut sebagai *protocol*
  atau kontrak)

- Hai, saya objek stack. Anda bisa minta saya untuk:
  - memberi tahu nilai “top”,
  - menambahkan item ke *stack* secara LIFO,
  - mengambil item yg ada di “top”,
  - memberi tahu apakah *stack* kosong.

Objek: Identitas, *State*, *Behavior*, *Responsibility*

- **Identitas**: setiap objek berbeda, meskipun *state* sama
- ***State***: informasi internal yang dimiliki objek
- ***Behavior***: operasi yang dapat dilakukan objek
- ***Responsibility***: aturan & kewajiban yang harus dijaga objek

Berkenalan dengan C++

C++ = C with class?

- Dari C (prosedural)
  - Data dan fungsi terpisah
  - Representasi data terlihat oleh seluruh program
  - Konsistensi tergantung disiplin pemrogram
- Menuju C++ (orientasi objek), oleh Bjarne Stroustrup (AT&T Bells Lab) ~1980
  - ***Class***: menggabungkan *state* dan *behavior*
  - ***Encapsulation***: representasi disembunyikan
  - ***Interface*** vs. implementasi: kontrak stabil, realisasi bisa berubah
- C++ tidak sekadar “C yang ditambah fitur”. C++ dirancang untuk memungkinkan pemodelan objek secara eksplisit.

Contoh Perbedaan dengan C

- *Typecasting* dalam C++ dapat dipandang sebagai fungsi.
  e.g., `(int) x` vs. `int(x)`
- *Function name overloading*: fungsi dengan nama yang sama namun dengan *signature* yang berbeda.
- Nilai default pada parameter formal.
  - e.g., `int f(int a, int b=2) { ... }`
    jika dipanggil dengan `f(5)`, `a` bernilai `5` dan `b` bernilai `2`.
- *Template function, operator function, inline function.*
- *Reference variable*, dan *call by reference* (berbeda dengan *address of*).
- Operator baru (e.g., `::`, `new`, `delete`), *keyword* baru (e.g., `class`, `private`).

C++: Reference Variable

- *Reference variable*: nama alias terhadap variabel tsb.
  - *Reference* tidak dapat direset untuk mengacu objek lain
  - Pendefinisian *reference variable* harus diinisialisasi dengan variabel lain

```
Stack  aStack = ...;        // inisialisasi objek
Stack& theStack = aStack;   // dua nama, satu objek

Stack& theStack{aStack};    // alt. sintaks C++ modern
```

- Contoh kegunaan *reference*: untuk *call-by-reference* dan *return value* dari sebuah fungsi

Kompatibilitas dengan C

- *Reference* **berbeda** dengan pointer
- Dalam C++ simbol & digunakan dengan 2 makna: *address-of* dan *reference*

```
int* py;
int& yr; // error (tidak diinisialisasi)

int y;
py = &y; // py akan berisi alamat dari y
```

Kompatibilitas dengan C

- Program C yang dikompilasi oleh C++ tidak dapat menggunakan kata kunci dari C++ sebagai nama *identifier*
- Dalam C++, setiap fungsi harus dideklarasikan (harus memiliki *prototype*).
  - Namun, sebagai *best practice*, deklarasi fungsi juga dianjurkan dalam C.
- Dalam C++, fungsi yang bukan bertipe `void`, harus memiliki instruksi `return`.
  - Dalam kebanyakan *compiler* C, fungsi non-`void` tanpa `return` hanya mengakibatkan *warning*.
- Perbedaan penggunaan *array* karakter dan *string*
  - C++ → PBO, hindari penggunaan *array* mentah

Konsep Kelas
& Kelas di C++

Class

- ***Class* adalah abstraksi desain**
- *Class* mendefinisikan:
  - tanggung jawab objek (via deklarasi *public function members / methods*)
  - perilaku yang tersedia (via definisi *function members / methods*)
  - state yang diperlukan (via deklarasi *private data members / fields*)
- *Key concept*:
  - Class adalah keputusan desain tentang *bagaimana objek akan hidup*.
- Peran: perancang kelas dan pengguna kelas.
  - **Perancang**: menentukan *interface* / kontrak & representasi internal objek.

Struct vs. *Class*

***Struct*:**

- Kumpulan data
- Tidak menjaga konsistensi sendiri
  - Setiap *field* bebas diakses dari luar.

***Class*:**

- Pemilik perilaku
  - Data sebagai konsekuensi untuk memenuhi kontrak perilaku
- Menjaga *invariants*

Dua jenis *member* / anggota dalam *class*:

- ***Function member*:** operasi (*service*/***method***) yang dapat diterapkan thd objek
- ***Data member*:** merupakan representasi internal dari kelas (atribut/***field***)

Kontrak Mendahului Data

- Desain *class* dimulai dari:
  - operasi yang dibutuhkan (kontrak)
- *Field* muncul karena:
  - diperlukan untuk menjalankan operasi / memenuhi kontrak
- Data tanpa *behaviour* ≠ objek

Struct vs. Class

Struct:

```
typedef struct {
    int x;
    int y;
} Point;

void moveTo (Point&) { ... }
```

Class:

```
class Point {
  public:
    void moveTo () { ... }
  private:
    int x;
    int y;
};
```

Pada class:

- Pengaturan akses terhadap anggota kelas: `private`, `public`, `protected`
- Perhatikan perubahan parameter aktual pada prosedur / *method* `moveTo`

Pengaturan Akses Member

Wilayah Deklarasi	Makna
`public`	Dapat diakses oleh fungsi di luar kelas dengan menggunakan operator selektor (`.` atau `->`) “Fungsi luar”: fungsi yang bukan anggota kelas tersebut
`private`	hanya dapat diakses oleh fungsi kelas tersebut
`protected`	seperti `private`, namun dapat diakses oleh kelas turunan

Clean OOP:

- *Field* selalu `private`.
- *Method* yg merupakan bagian dari kontrak → `public`, selain itu → `private`.

Contoh Desain: Stack sebagai Objek

- Tanggung jawab *stack*:
  - menyimpan elemen
  - menjaga urutan LIFO

- Kontrak:
  - peek
  - push item
  - pop
  - empty?

- *Invariants*:
  - tidak pop saat kosong
  - tidak push saat penuh

Deklarasi Kelas Stack + definisi method

```
#include <vector>

class Stack {
  public:
   // deklarasi method (prototype):
      int  peek() const;  // `const`: method ini tidak memutasi
objek
      void push(int);
      int  pop();
   // deklarasi + definisi method di dalam class body:
      bool empty() const { return data_.empty(); }
  private:
   // field
      std::vector<int> data_; // hindari array mentah spt `int
*data_;`
}; // perhatikan titik koma!
```

Pendefinisian method di luar Class

```
/* definisi peek, push, dan pop di luar class body */
#include <stdexcept>

int Stack::peek() const {
  if (empty()) {
    // tangani error
    throw std::runtime_error("peek() dari stack kosong");
  } else { return data_.back(); }
} // tidak perlu titik koma!

void Stack::push (int item) {
  data_.push_back(item);
}
```

Pendefinisian method di luar Class

```
int Stack::pop() {
  if (empty()) {
    throw std::runtime_error("pop() dari stack kosong");
  } else {
    int v = data_.back();
    data_.pop_back();
    return v;
  }
}

// penanganan error dengan `throw` dibahas di lain waktu
```

Pointer implisit “this”

- *Method* dimiliki class
- *Field* dimiliki objek
- `this` adalah *pointer* implisit yang dimiliki setiap *method* untuk menunjuk objek yang:
  - menerima pesan
  - menjalankan perilaku
- Seolah didefinisikan sebagai:

```
X* this
```

- Contoh penggunaan:

```
void Stack::push (int item) {
   this->data_.push_back(item);
}
```

- Manfaat: Jika terjadi *name shadowing*, `this` membuat eksplisit *identifier* mana yang dimaksud
  - e.g., jika ada variabel `data_` dalam *method* `push()`, tapi ada juga *field* `data_` di kelas `Stack`.

```
x->y
```
adalah alt. sintaks untuk
```
(*x).y
```

Objek, Instansiasi Class

- Objek: entitas konseptual
- Variabel: cara mengakses objek
- Banyak variabel → satu objek
- Satu variabel tidak selalu sama dengan satu objek

```
Stack myStack;

std::array<Stack, 10> myStacks;
  // sekumpulan `Stack`

Stack yourStack{myStack};
  // menciptakan salinan dari
`myStack`

Stack& mijnStack{myStack};
  // nama kedua untuk objek yg sama

myStack.push(10);
  // mengirim pesan push `10` kpd
objek

int x = myStack.pop();
  // mengirim pesan `pop` kpd objek
```

Penulisan Kode Kelas

- Kode untuk kelas terdiri dari:
  1. *Interface / specification*: **deklarasi** kelas. Contoh jika nama kelas: Abc.
     Dituliskan ke dalam file **`abc.h`** atau **`abc.hpp`**.
  2. *Implementation / body* : **definisi** dari method-method dari kelas tersebut.
     Dituliskan ke dalam file **`abc.cc`** atau **`abc.cpp`**.
- Untuk mencegah penyertaan *header* lebih dari satu kali, deklarasi kelas dituliskan di antara `#ifndef ABC_HPP` dan `#endif`
  - Alternatif modern tanpa `#ifndef`, `#define`, dan `#endif`: cukup tulis
    `#pragma once`
    di baris pertama program

Contoh *Header File*

```
// Nama file: stack.hpp
// Deskripsi: interface dari kelas Sta
ck

#ifndef STACK_HPP
#define STACK_HPP

#include <...>

class Stack {
  // ... (sesuai contoh sebelumnya)
};

#endif
```

PERHATIAN: Di dalam *header file*, jangan menuliskan **definisi** objek/variabel karena akan mengakibatkan kesalahan "*multiply defined name*" pada saat *linking* dilakukan.

Contoh *Header File* (Modern C++)

```
// Nama file: stack.hpp
// Deskripsi: interface dari kelas Sta
ck

#pragma once


#include <...>

class Stack {
  // ... (sesuai contoh sebelumnya)
};
```

PERHATIAN: Di dalam *header file*, jangan menuliskan **definisi** objek/variabel karena akan mengakibatkan kesalahan "*multiply defined name*" pada saat *linking* dilakukan.

Contoh *Implementation File*

```
// Nama file: stack.cpp
// Deskripsi: implementasi/body dari kelas Stack

#include "stack.hpp"

Stack::Stack() {
  // ... implementasi ctor
}

Stack::~Stack() {
  // ... implementasi dtor
}

// dst ... (sesuai contoh sebelumnya)
```

Pemanfaatan Kelas

- Perancang kelas:
  1. Kompilasi *file* implementasi (misal abc.cpp) menjadi kode objek (abc.o)
  2. Berikan *file header* (abc.hpp) dan file kode objek (abc.o) kepada pemakai kelas (mungkin dalam bentuk *library*)
- Pemakai kelas:
  1. Sertakan *file header* di dalam program (main.cpp)
  2. Kompilasi program C++ menjadi kode objek (main.o)
  3. Link main.o dengan abc.o yang diberikan perancang kelas

Next Topic

- Siklus hidup objek
- Dalam C++: 4 sekawan:
  - ctor, cctor, dtor, operator=